



University of
Southern Denmark

LARGE-SCALE DRONE PERCEPTION

DRONE MASTER

2. SEMESTER

Pumpkin counting miniproject

made by:

Adam Kolodziej adkol23@student.sdu.dk

Lucas R. Jeppesen ljepp19@student.sdu.dk

Oliver Torngaard oltor19@student.sdu.dk

June 1, 2026

Contents

1	Introduction	3
1.1	Overall goal of the miniproject	3
2	Github	3
3	Color segmentation	3
4	Generating an orthomosaic	5
4.1	Loading images and setting up the project	5
5	Calculating objects	7
6	splitting orthomosaic into tiles	9
6.1	counting with the overlap	9
7	End Notes	10

List of Figures

1	Original orthomosaic	3
2	RGB and CieLAB masks	4
3	RGB and CieLAB graphs	5
4	Dense reconstruction (pointcloud model)	6
5	Digital elevation model (Digital surface model)	6
6	Cropped orthomosaic	7
7	Image after process stages (a) original image (b) after masking	7
8	Image after process stages (a) erosion (b) dilatation	8
9	Centers of contours	9
10	Caption	9
11	Pumpkin examples	10

1 Introduction

1.1 Overall goal of the miniproject

The main goal of the project is to calculate number of pumpkins which are grown on the field. Every operation is based on generated orthomosaic which was created using WebODM. Next, we cropped it to limit the data to process. Later we wrote a code that splits the orthomosaic into tiles. The final step was to perform proper pumpkin recognition and calculate how many of them were located in the area.



Figure 1: Original orthomosaic

2 Github

A git repository on GitHub was created to work on the project. The final version of the pumpkin counter can be found in the file `Orthomosaic_count/Exercise3.4.3_5.py` it assumes there is an orthomosaic in the folder `orthomosaics` named `pumkin_filed.tif`

the repo can be found here: <https://github.com/u000000/Pumpkin-counting-miniproject.git>

3 Color segmentation

When color segmenting the map, a test image where some of the pumpkins throughout the field were annotated using a pure red color, to extract the color and image data of the pumpkins for use in the

assignments. A pure red color was chosen since it would be easily isolated when creating a mask, since there were no other areas in the map with a pure red color. The mask isolating the annotated pumpkins was created using OpenCV, using the InRange function. The InRange function would check the chosen image for colors within two given threshold values and isolate all pixels of said color. For this project the annotated pumpkins were isolated and a mask shown in figure 2a was created. To calculate the mean and standard deviation of the pumpkins, the generated mask was used in the OpenCV function meanStdDev. For the RGB color spectrum, the results were as shown in figure 2b, which are in BGR, since OpenCV works with BGR notation. For CIE LAB the image color was converted using the OpenCV function cvtColor, and meanStdDev were used to find mean and standard deviation. The RGB mean was [128.46463142] [218.56515264] [252.70513775], while the standard deviation was [21.41585675] [25.13901853] [8.69158983]. For CIE LAB the mean was [225.69843634] [128.99106478] [176.34921817], while the standard deviation was [17.55660703] [10.69080414] [9.59233129].

To have a comparison of segmentation methods, the InRange of CIE LAB and the euclidean distance of RGB space were calculated. For the InRange of CIE LAB, a new mask was found, which gave the mask shown in figure ??, while the euclidean max and min distance between the annotated pumpkins and the standard color code for orange. The minimum distance was 138.25 while the maximum distance was 334.41. For this project, it was decided that the CIE LAB segmentation would be used in the future assignments, since CIE LAB gave better results when looking for pumpkins, as can be seen in figure 3b where the color plane was more even when compared to RGB in figure 3a.

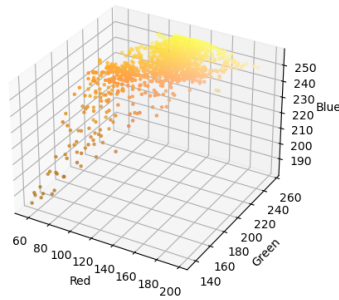


(a) Mask RGB

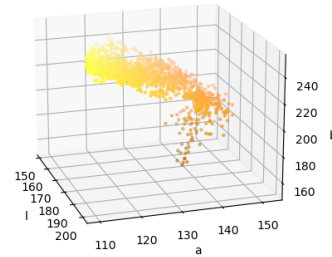


(b) CIE LAB Mask

Figure 2: RGB and CIE LAB masks



(a) BGR graph



(b) CieLAB graph

Figure 3: RGB and CieLAB graphs

4 Generating an orthomosaic

We decided to use WebODM because it seems to be quite simple to use.

The process described in this paragraph is mostly based on the provided tutorial.

4.1 Loading images and setting up the project

When we have a project created, the first step is loading images. We do it by clicking "Select Images and GCP", next we setup parameters. We set the "orthophoto resolution" parameter to 2.5 cm which gave us reasonable output.

Original Orthomosaic can be seen as figure 1 1 in the section 1.1. Cropping was performed in the QGIS. There are several ways to do it. We particularly followed these steps:

- create "New Shapefile Layer"
- write a name and change Geometry type to Polygon -> OK
- "Toggle editing" -> "Add Polygon"
- Saved masked area using "Raster" -> "Extraction" -> "Clip Raster by Mask Layer"
- choose input layer and mask layer and run
- cropped orthomosaic will show up as a new layer

It was important to limit the orthomosaic because we were supposed to calculate pumpkins only on the main field - not the one at the bottom.

Here is the data that was required in the project:



Figure 4: Dense reconstruction (pointcloud model)

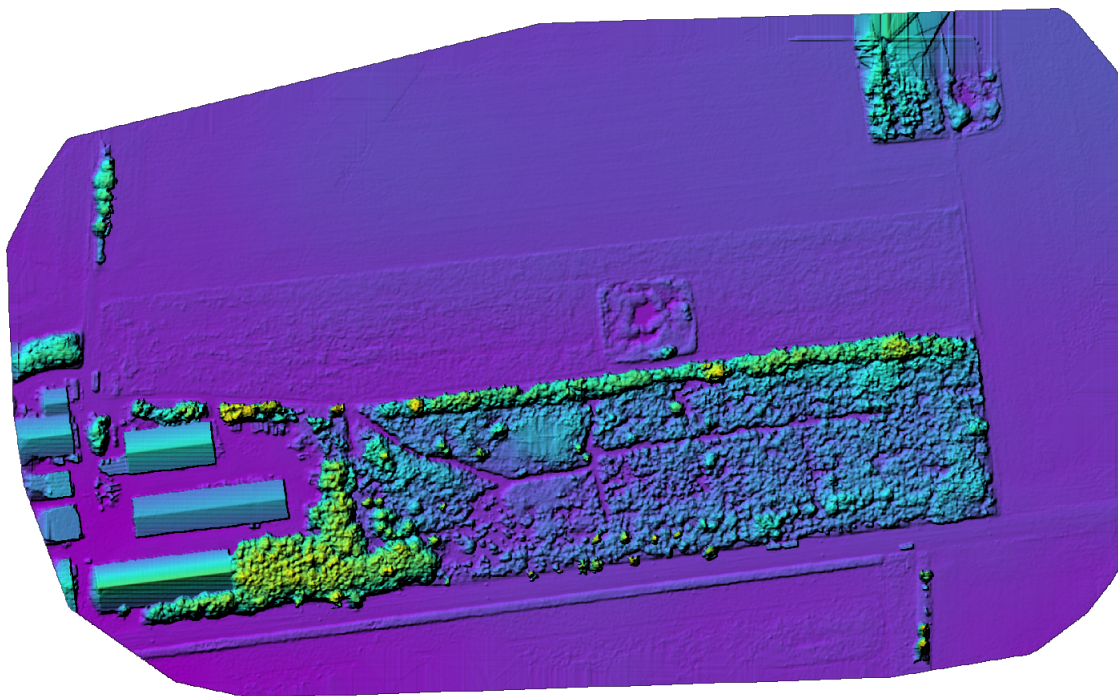


Figure 5: Digital elevation model (Digital surface model)

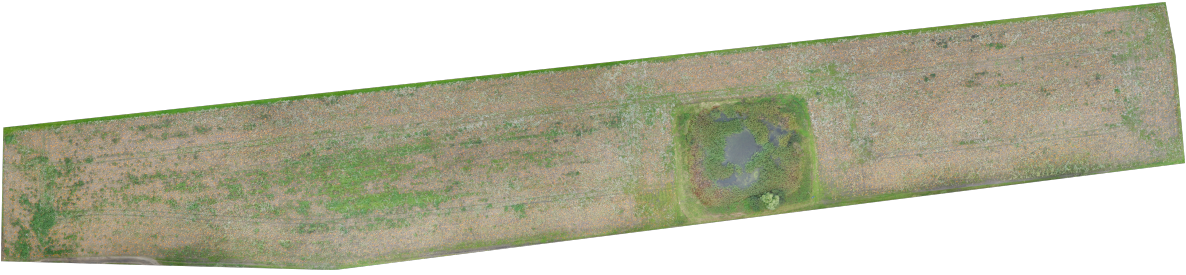


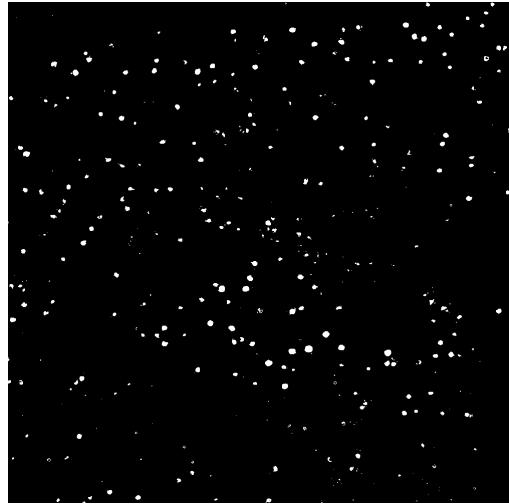
Figure 6: Cropped orthomosaic

5 Calculating objects

In this section, we will focus on computer vision. This process is performed on each tile separately. The counting part is placed in the "count_pumpkins(img)" function. The first provided image is converted to CieLab color space. This color space makes it easier to work on color detection. Next, we perform masking based on Euclidean distance. We checked that pumpkins' colors are spread in shape close to the "sphere" which is space that can be determined using Euclidian distance. We also tried inRange but the masked area is a cuboid which includes some unwanted colors). We wrote inEuclidianDist(img,mean,max_dist) function. As a parameter mean_color, information calculated during the color segmentation process is used.



(a)

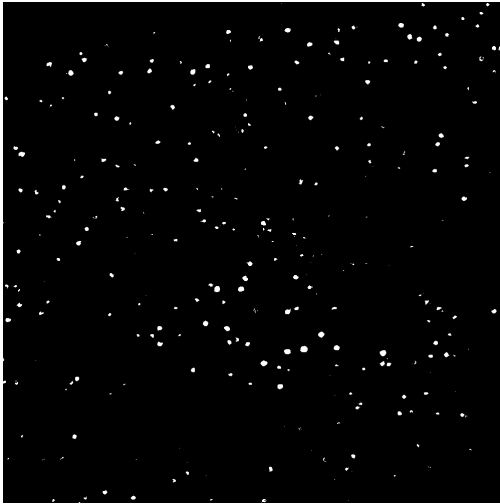


(b)

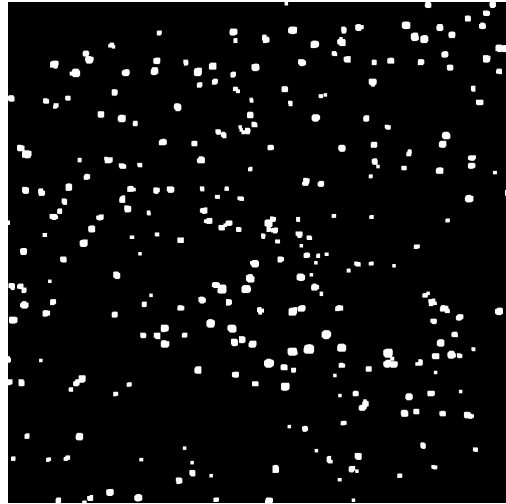
Figure 7: Image after process stages (a) original image (b) after masking

Masked image is subjected to erosion and dilation. Erosion is used to get rid of noise and fake pumpkins which were detected as small areas. Dilatation makes everything clearer and easier to work with. Both

operations work on an image using a specified kernel - square.



(a)



(b)

Figure 8: Image after process stages (a) erosion (b) dilatation

Next, the counting is performed. First, we look for contours using `findContours()`. Then we want to get the center of each contour. To do so we calculate the contour's center of mass, which was placed in an array. The final step is to iterate through the array of centers and calculate if they are located in the overlapping area or on the tile area. If they are in the overlap, we just ignore them. That way we get rid of pumpkins that have part of the body inside the tile but the centers of them are in the neighboring tile so they should be counted in that neighboring tile. It means that we won't calculate them twice.



Figure 9: Centers of contours

6 splitting orthomosaic into tiles

To help not use all our computer's memory on storing the full orthomosaic, we split the orthomosaic into tiles with some overlap. We added some padding on the orthomosaic, to allow for all the tiles to have the same size, and have the overlap extended over the side border of the orthomosaic, which is imported for the way we count the pumpkins, and handle the border zone between tiles. In figure 10 you can see a simplified tiling with two tiles the tiles are blue and green respectively.

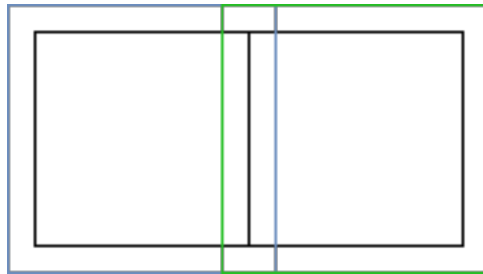


Figure 10: Caption

6.1 counting with the overlap

As it can be seen in figure 10 the blue and green areas overlap in the overlap area the pumpkins would be counted double if we do not consider it as seen in figure 11b. Removing the overlap will not help,

as we still can have pumpkins on the border between two tiles, which will count on both sides as seen in figure 11a. To fix this problem we define areas like the black one in figures 11b, and 11c. Then we calculate the center of the pumpkins, Depending on which area the centers are in we count it in that tile.

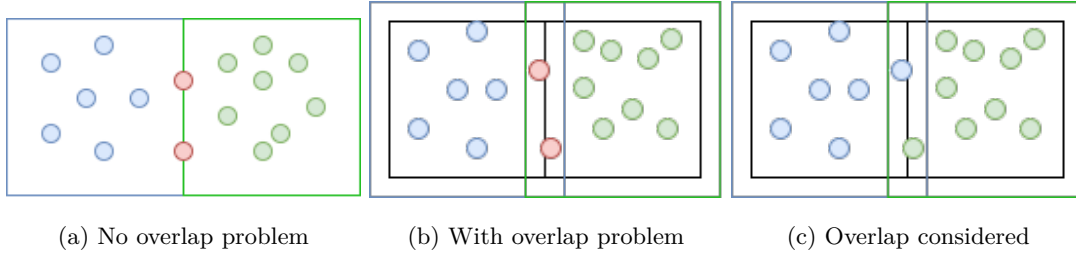


Figure 11: Pumpkin examples

7 End Notes

In this project a pumpkin counting algorithm has been made. The algorithm was able to correctly find and count the pumpkins of a given map of a field. The GSD for this project ended up being 2.2 cm and an average of 0.667358082 pumpkins were counted for each m^2 of the field. Sadly this algorithm would still need some development before it could be used by farmers, since it still requires a survey of the field to be done, the map to be generated by WebODM, and be placed in the correct folder with the correct file name. This would end up becoming a complex and time consuming task, that most farmers would rather find other alternatives to. In terms of correctly counting the number of pumpkins, the algorithm were able to find a total of 21878 pumpkins, which would give the farmers a good estimate on the total number of pumpkins, but would most likely not be accurate enough to give actual value to the farmer. This means that the algorithm should be further tuned before it could generate value to farmers.